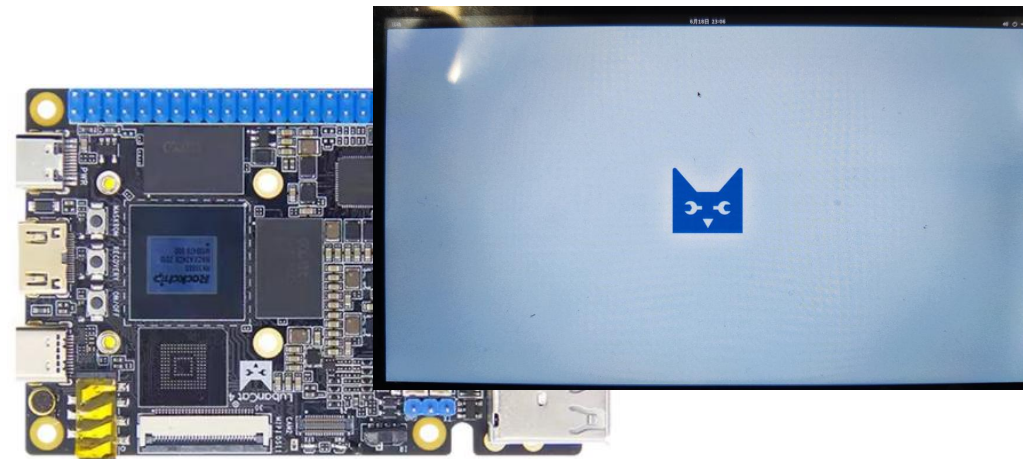
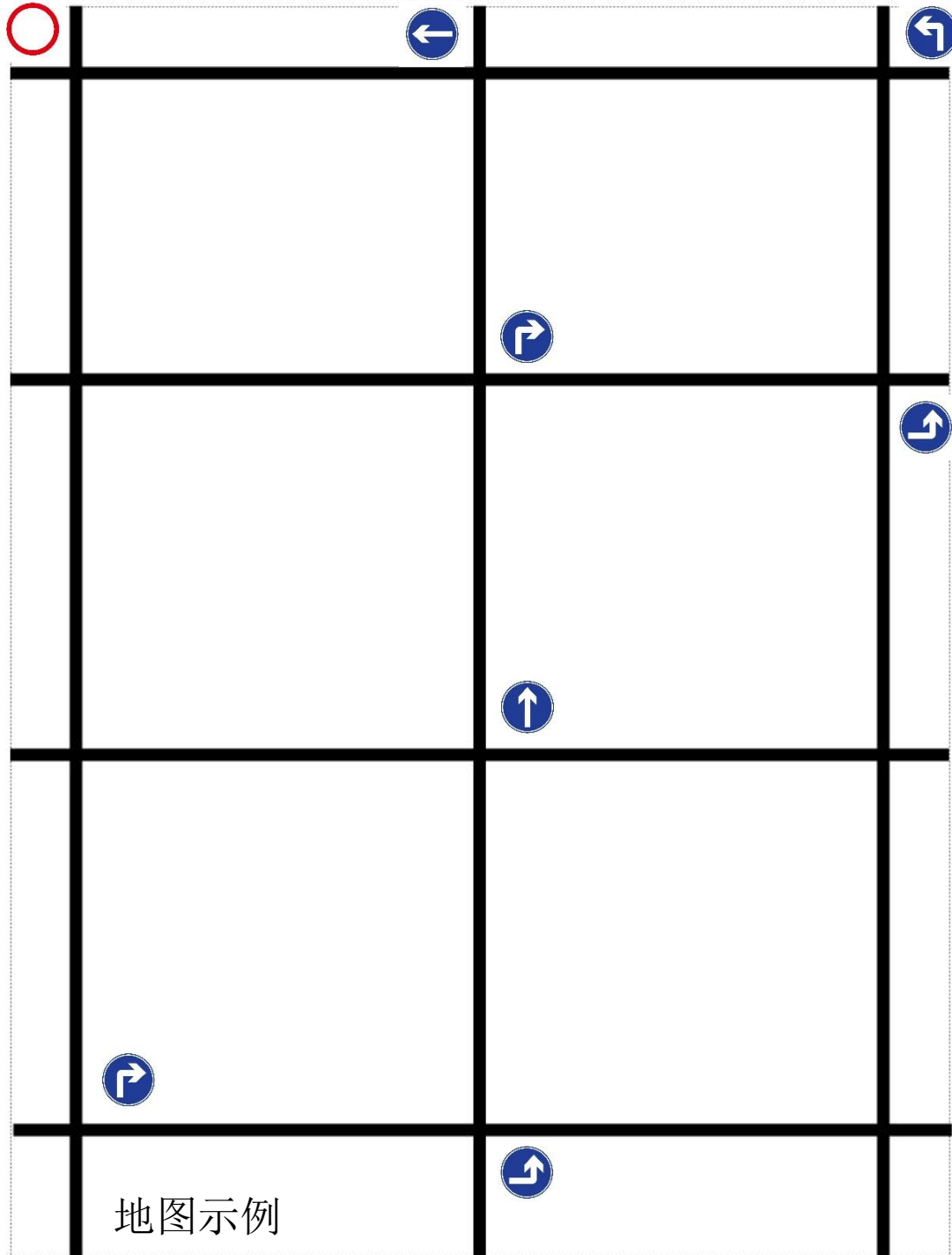
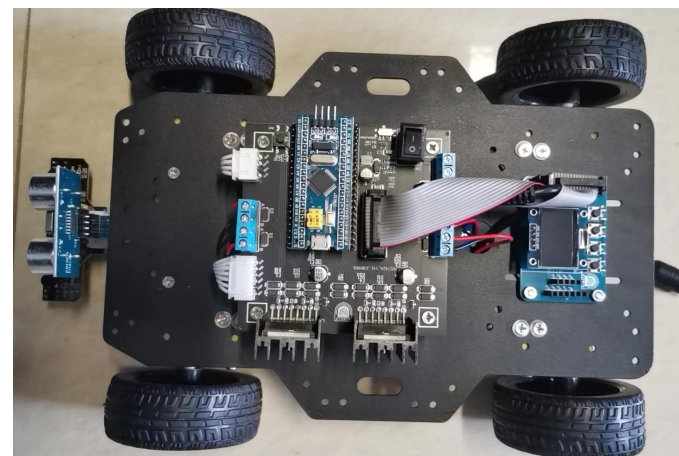




任务:

- RK3588开发板调用USB摄像头捕获图像
- 借助YOLO算法识别交通标志，给出结果。



- 小车依据结果执行直行、左转、右转、停车等，
- 按指示导航至目标终点停车，完成任务。

任务分解：

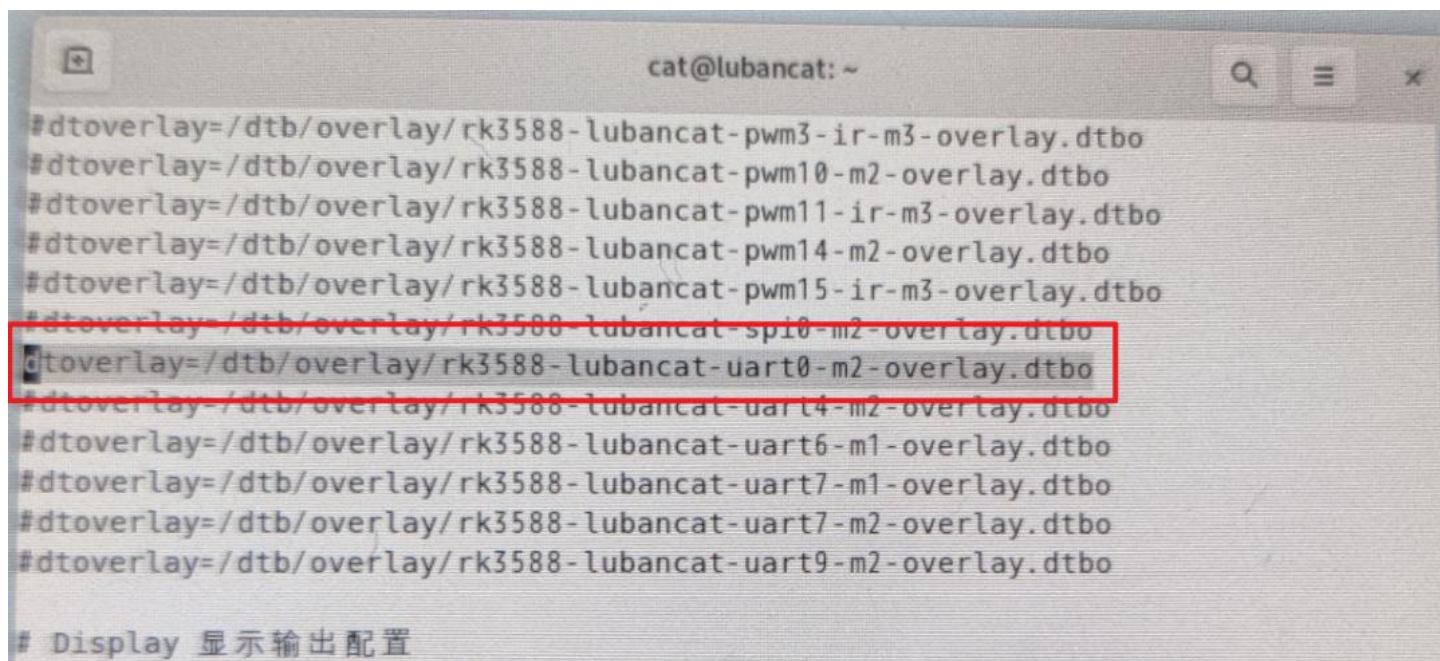
- I. **摄像头开启：** RK3588默认已经安装USB摄像头驱动程序，我们只需在RK3588平台安装opencv，程序里直接调用`cap = cv2.VideoCapture(0, cv2.CAP_V4L2)`即可打开摄像头。
- II. **摄像头采集图像：** 开启摄像头之后，调用 `ret, frame = cap.read()`从摄像头读取一帧图像。
- III. **识别图像：** 利用RK3588内置的NPU单元，加载已经训练好的交通标志识别模型对摄像头拍摄的图像帧进行识别。
- IV. **RK3588平台和STM32通信：** STM32检测到路口，通过串口给RK3588发送开启检测命令， RK3588调用摄像头开启检测并识别，将识别结果保存在数组里，判断连续10帧检测的结果是否一致，然后通过**串口**将识别结果发送给STM32单片机，从而控制小车导航。

UART口连接配置:

板卡上的UART0默认没有启用，需注释掉设备树节点或者设备树插件的加载，重启系统来开启UART0。

具体操作如下：

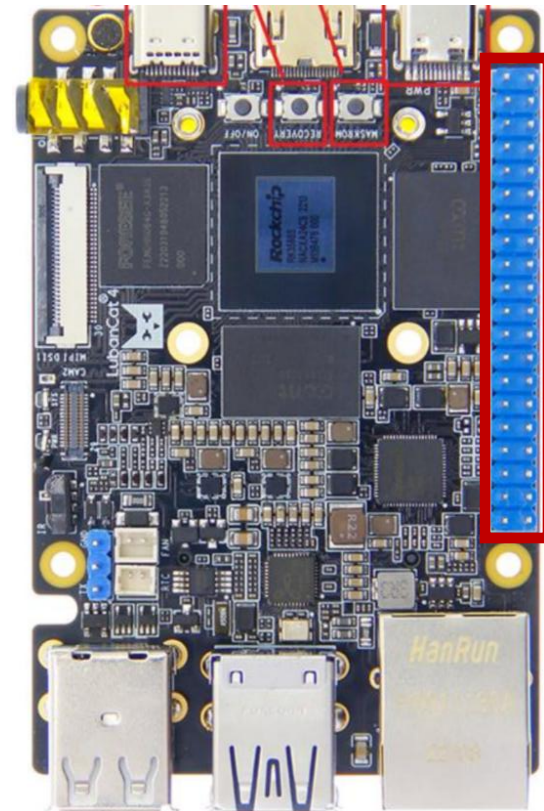
- 打开终端，输入：**sudo vi /boot/uEnv/uEnv.txt** 然后取消UART0前面的注释，启用UART0 如下图所示。



```
cat@lubancat: ~
#dtoverlay=/dtb/overlay/rk3588-lubancat-pwm3-ir-m3-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-pwm10-m2-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-pwm11-ir-m3-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-pwm14-m2-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-pwm15-ir-m3-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-spi0-m2-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-uart0-m2-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-uart4-m2-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-uart6-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-uart7-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-uart7-m2-overlay.dtbo
#dtoverlay=/dtb/overlay/rk3588-lubancat-uart9-m2-overlay.dtbo

# Display 显示输出配置
```

- 修改后按下 Esc 键，输入 **:wq** 保存并退出，再重新启动系统生效配置。



UART口连接配置:

RK3588板卡和STM32单片机进行UART0通信需要连接三根线:

RK3588板卡	STM32
GND	GND
TX	RX
RX	TX

安装串口依赖项periphery, 终端输入:
`sudo pip3 install python-periphery`

导入库: `from periphery import Serial`

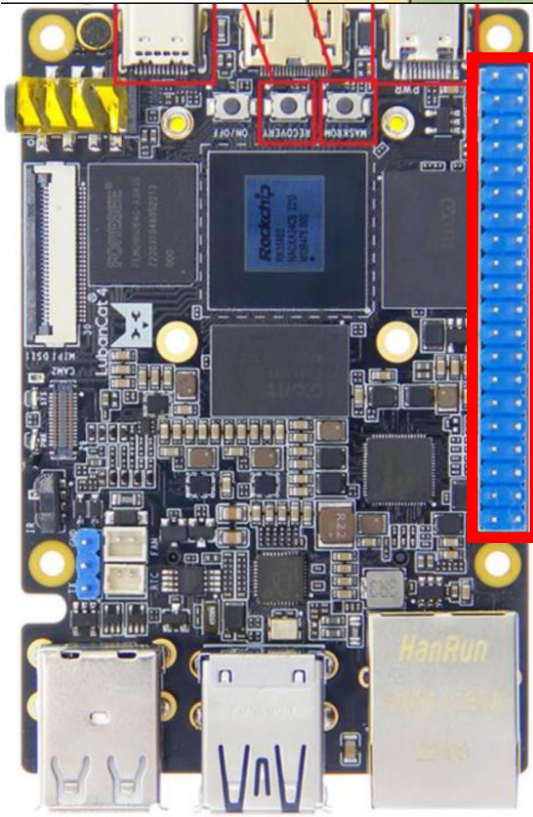
```
# 打开串口
serial = Serial(
    "/dev/ttyS0",
    baudrate=115200,
    databits=8,
    parity="none",
    stopbits=1,
    xonxoff=False,
    rtscts=False,
)
```

```
# 接收下位机发送的命令
command = serial.read(5)

# 将检测结果通过串口发送给下位机
try:
    serial.write(detected_labels[4].encode())

    serial.write(b"python-periphery!\n")
```

LuBanCat4引脚图							
通用功能	编号	GPIO	物理引脚		GPIO	编号	通用功能
		3.3V	1	2	5V		
I2C5_SDA_M3	47	GPIO1_B7	3	4	5V		
I2C5_SCL_M3	46	GPIO1_B6	5	6	GND		
	32	GPIO1_A0	7	8	GPIO4_A3	131	UART0_TX_M2
		GND	9	10	GPIO4_A4	132	UART0_RX_M2
	33	GPIO1_A1	11	12	GPIO1_D6	62	PWM14_M2
	39	GPIO1_A7	13	14	GND		
	40	GPIO1_B0	15	16	GPIO3_C1	113	
			17	18	GPIO3_D2	122	
		B2	19	20	GND		
		B1	21	22	GPIO3_D4	124	
		B3	23	24	GPIO1_B4	44	SPI0_CS0_M2
			25	26	GPIO1_B5	45	SPI0_CS1_M2
		B0	27	28	GPIO4_B1	137	I2C6_SCL_M3
		A6	29	30	GND		
		B7	31	32	GPIO1_D7	63	PWM15_IR_M3
		D3	33	34	GND		
		D5	35	36	GPIO4_A0	128	
		C0	37	38	GPIO4_A1	129	
			39	40	GPIO4_A2	130	



摄像头配置：

安装opencv，打开终端输入： `sudo apt-get install python3-opencv`

程序里调用： `import cv2`

示例教程：[摄像头使用-Opencv](#)

大致流程：

```
# 打开默认摄像头 (索引0)
cap = cv2.VideoCapture(0)
```

改为 `cap = cv2.VideoCapture(0, cv2.CAP_V4L2)`

```
# 用于存储检测结果的列表
detected_labels = []
```

```
# 从摄像头读取一帧图像
ret, frame = cap.read()
```

```
# 实时预处理
img, ratio, (dw, dh) = letterbox(frame, new_shape=(640, 640))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# 实时推理
outputs = rknn_lite.inference(inputs=[img])
```

```
# 实时后处理
boxes, classes, scores = yolov5_post_process(outputs)
```

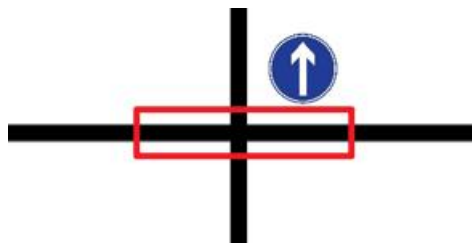
```
# 添加检测到的标签到列表中
if classes is not None:
    detected_labels.append(CLASSES[classes[0]])
```

通过以上步骤，检测结果就被添加到detected_labels数组中，等待STM32发送命令查询

串口和摄像头联合使用：

大致流程：

- I. 小车运行过程中，遇到路口时，四个红外传感器全部检测到黑线，此时，可以当作检测信号标志位。
- II. 先让小车停下，通过串口给RK3588发送一个开始识别标志 (如‘begin’)。
- III. RK3588接收到命令后，启动识别程序，将识别结果再通过串口发送给STM32单片机（如‘turn_right’）。
- IV. STM32根据收到的命令控制小车，最终到达终点。



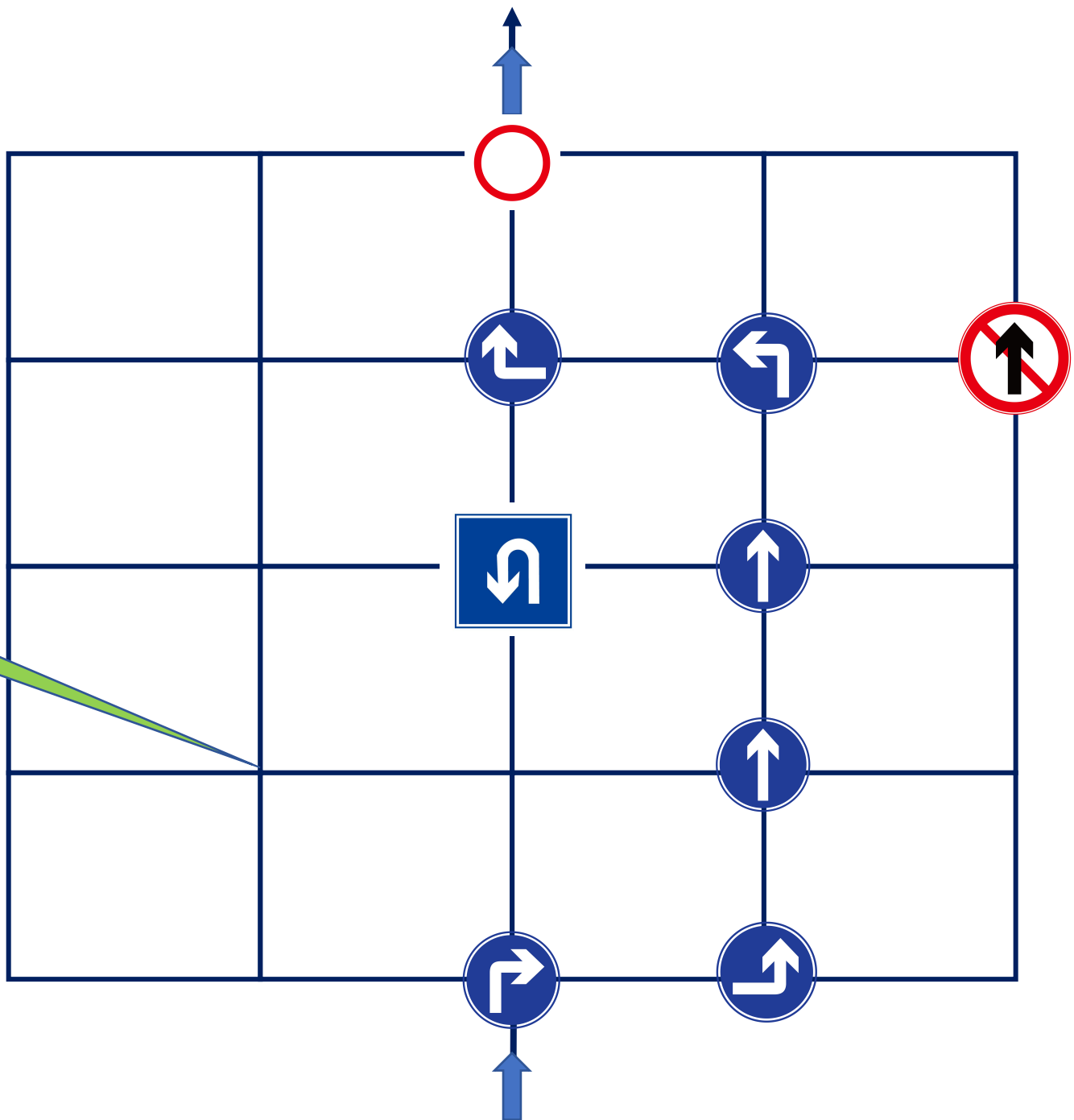
提供主程序编写思路：

```
if __name__ == '__main__':
```

- a) 创建RKNNLite对象并加载RKNN模型。
- b) 初始化运行时环境。
- c) 打开串口与下位机连接。
- d) 打开默认摄像头。

```
while True:
```

- a) 在无限循环中，首先监听串口获取命令；
- b) 当接收到特定命令时，清空已检测的标签列表，并开始处理接下来的10帧图像。
- c) 对每帧图像进行预处理（调整大小并填充），转换颜色空间。
- d) 使用RKNNLite进行实时推理。
- e) 对推理结果进行后处理，得到边界框、类别和置信度。
- f) 在帧上绘制检测到的边界框并显示图像。
- g) 将后面连续6帧中稳定出现的同一类别的标签添加到detected_labels列表中。
- h) 当检测到连续6帧的标签一致时，将该标签通过串口发送给下位机。



无标识路口
可任意行走